Generative Compression for Differentiable Opaque Meshes

Technical report

November 5, 2026

Abstract

MeshSplatting produces an opaque, engine-ready triangle mesh with per-vertex spherical harmonics (SH), but its FP32 output is 430–740 MB per scene. We keep MeshSplatting's optimisation untouched and add two training-time decisions and one delivery codec: triangles are split only when the distortion they remove is worth the bytes they add, the same price is applied in reverse as an edge-collapse prune, and the trained mesh is encoded into a single file whose per-vertex SH is replaced by shared dictionary indices. One rate–distortion parameter λ traces the whole curve. On Mip-NeRF 360 the 642 MB FP32 mesh becomes 26.6 MB at -0.24 dB ($24\times$), 9.0 MB at -0.92 dB ($72\times$) and 4.3 MB at -1.68 dB ($151\times$); the delivered asset is a standard mesh that decodes in 2–6 s on one CPU core and renders at 145–350 fps on a phone.

1 Methodology

We keep MeshSplatting's optimisation untouched and change two things: *which* triangles get split during growth (priced in bytes), and a reverse prune under the same price. The trained mesh then goes through a codec that replaces per-vertex SH by shared dictionary indices. The vertex count is no longer a hand-set cap but an output of one rate–distortion trade-off λ. Figure 1 shows where the pieces sit.

1.1 Where we intervene in MeshSplatting

Upstream training has two stages. **Stage 1 (iterations 0–10 000)** grows a soft triangle soup: every 500 iterations a fixed fraction of triangles (`add_percentage`) is split, chosen by importance sampling, up to a hand-set cap. At iteration 11 000 a restricted Delaunay rebuild re-triangulates the soup. **Stage 2 (11 000–30 000)** optimises the now-opaque mesh (opacity schedule, $2\times/4\times$ supersampling after 20k/25k).

Our changes sit only in the growth decisions:

1. **Stage 1, the split rule.** The importance-sampled split is replaced by a priced one: a triangle is split only if the distortion it would remove is worth the bytes it adds (Section 1.2). Growth stops by itself when no triangle passes.
2. **Stage 2, λ-prune rounds** at iterations 12k / 14k / 16k / 18k (after the Delaunay rebuild, before supersampling): edges whose faces carry too little residual for the bytes they cost are collapsed, at most 15 % of the vertices per round, with a fold-over guard. Same λ, applied in reverse.

Losses, learning rates, opacity schedule, Delaunay rebuild, supersampling and final cleaning are upstream code imported as is. The appearance optimisation is MeshSplatting's own.

1.2 How the rate cost is estimated and used

Distortion. For every triangle t we accumulate during each densification interval its **distortion mass** ΔD_t: the per-pixel L1 error summed over the pixels whose front-most triangle is t (from the rasteriser's per-pixel triangle ids), divided by the megapixels of the image and averaged over the

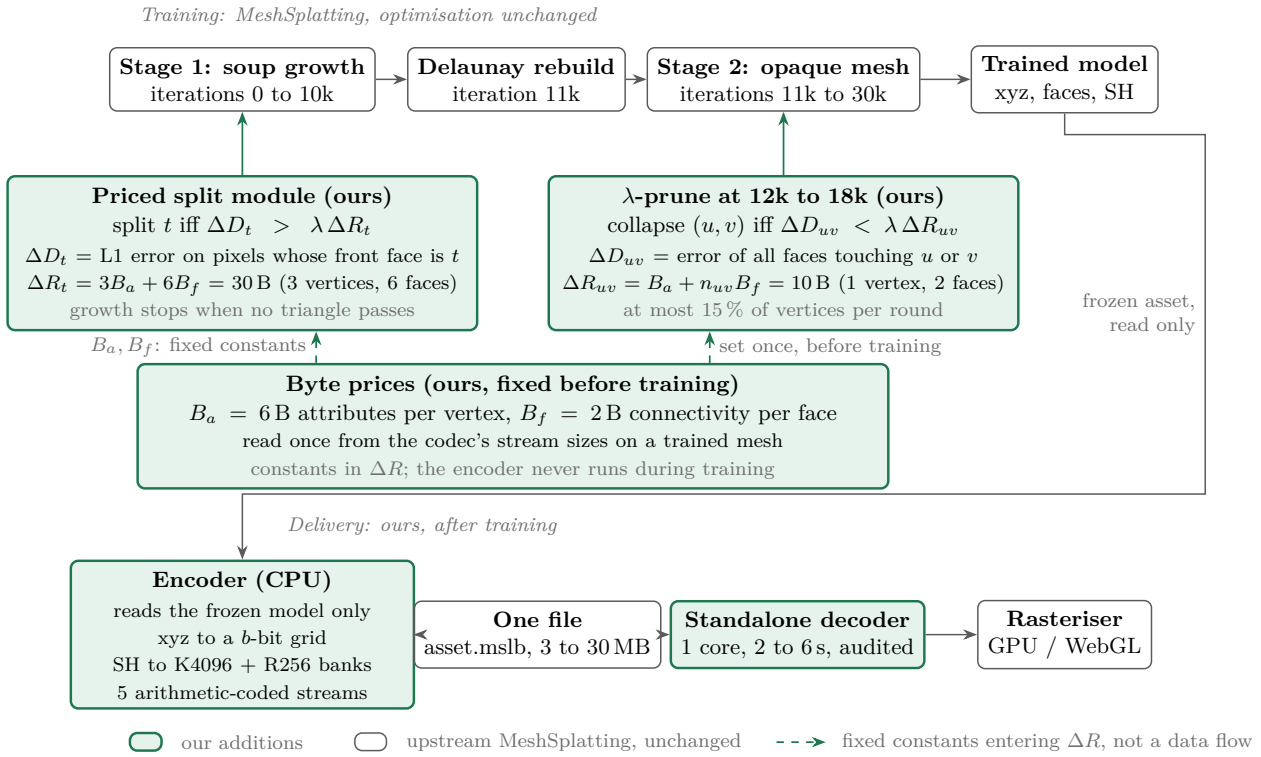


Figure 1: Pipeline: two priced decisions during training, one dictionary codec at delivery. Grey boxes are MeshSplatting as published; tinted boxes are ours. The byte prices are two constants read once from the codec's stream sizes; the encoder itself runs only on the frozen, finished model and cannot influence how the mesh grew.

training views of the interval. A triangle that is visible in more views, covers more pixels, or is rendered worse carries more mass. The prune uses the same per-triangle quantity ΔD_t, summed over the triangles around an edge.

Rate. Rate is counted in bytes of the delivered file, from two constants:

- $B_a = 6$ B, the *attribute* bytes of one vertex: quantised position residual (≈ 2.3 B), DC colour (≈ 1.5 B) and its two SH dictionary indices (≈ 2.4 B);
- $B_f = 2$ B, the *connectivity* bytes of one face.

A file with V vertices and F faces therefore costs $VB_a + FB_f$ bytes plus a fixed 0.6 MB of dictionaries and header. Neither constant contains the other. The constants are not learned: they are the measured stream sizes of our own lossless coder (Section 1.3) on one delivered Room asset, divided by its vertex and face counts (positions 2.3 B, DC 1.5 B, two dictionary indices 2.4 B per vertex; connectivity 31.7 bit per vertex ≈ 2 B per face at two faces per vertex), fixed before training and reused for every scene and every λ. The true cost does drift with the operating point: over the 44 delivered assets it is 9.5–13.3 B per vertex (median 10.8 against the 10 B the constants imply), rising slightly with λ because sparser meshes have longer edges and fewer edge-cache hits. Since the rules below compare $\Delta D/\Delta R$ with λ, a price error of a factor c is identical to using $c\lambda$; the observed ± 30 % is less than a quarter of one step of our λ grid, so it moves the operating point slightly along the same curve rather than changing the curve.

The two rules. Every mesh edit is a discrete decision in the Lagrangian $J = D + \lambda R$ and is accepted iff it lowers J:

$$\text{split } t \iff \frac{\Delta D_t}{\Delta R_t} > \lambda, \qquad \Delta R_t = 3B_a + 6B_f, \tag{1}$$

$$\text{collapse } (u,v) \iff \frac{\Delta D_{uv}}{\Delta R_{uv}} < \lambda, \qquad \Delta D_{uv} = \sum_{f \in N(u) \cup N(v)} \Delta D_f, \quad \Delta R_{uv} = B_a + n_{uv} B_f, \tag{2}$$

where $N(u)$ is the set of faces that have u as a corner and n_{uv} the number of faces that contain the edge (u, v).

- **Split.** Subdividing t at its three edge midpoints adds three vertices. The stage-1 faces are discarded at the Delaunay rebuild; for a closed triangle mesh Euler's formula gives $F \approx 2V$, and our delivered meshes have 1.8–3.4 faces per vertex (median 2.3), so each added vertex is charged two faces: $\Delta R_t = 3B_a + 6B_f = 30\,\text{B}$. ΔD_t is an upper bound on the error a split can remove.
- **Collapse.** One collapse merges u and v into their midpoint: one vertex and the n_{uv} faces on the edge disappear ($n_{uv} = 2$ for an interior edge, 1 on a boundary), so $\Delta R_{uv} = B_a + n_{uv} B_f = 10\,\text{B}$ for an interior edge. ΔD_{uv} is the error of every face whose shape the collapse changes, i.e. the union of the two vertex rings; a ring with little error is one the remaining optimisation can repair. A prune round applies many collapses, in increasing order of $\Delta D_{uv}/\Delta R_{uv}$, on an independent set of edges (no two share a vertex), skipping collapses that would fold a surviving face, up to 15 % of the vertices per round.

Figure 2 works through one split and one collapse with numbers. λ is in units of L1 mass per byte. Four values (6.2×10^{-4}, 1.85×10^{-3}, 6.2×10^{-3}, 1.85×10^{-2}) trace the RD curve; one λ stops growth at 15–100 % of the dense vertex count depending on the scene, i.e. the budget adapts to scene content.

Choosing λ. λ is the Lagrange multiplier, as in the λ-mode rate control of video codecs and in HAC / RDO-Gaussian: it fixes the marginal slope, not the size, and the size decreases monotonically with λ. We have no closed form; empirically log(size) is close to linear in $\log \lambda$ with slope ≈ -0.6 on every scene (Table 2: room 22.1 → 2.9 MB, garden 36.4 → 5.0, kitchen 24.5 → 3.8 over a 30× range of λ), so a target size is reached by one interpolation on that law or one short calibration run. The alternative, stopping growth at a byte budget, is our "train-at-budget" ablation (Section 2.5); it hits a size exactly but gives each scene a different marginal slope, whereas one λ is the same exchange rate everywhere.

Why constants. B_a and B_f were calibrated once on one asset and then frozen: they are inputs to training, not outputs of it. Measured afterwards on all 44 delivered assets the true cost is 9.5–13.3 B per vertex against the 10 B the constants imply, and a price error of a factor c is equivalent to using $c\lambda$, i.e. it shifts the operating point along the curve by less than a quarter of a grid step. A per-scene price would buy nothing but would make λ a different exchange rate in every scene.

The encoder is not in the training loop. Its only contribution to training is these two constants; ΔR is computed from them, never by encoding anything during training. The encoder itself runs once, after iteration 30k, on the frozen model, and cannot influence how the mesh grew.

1.3 Encoder and decoder

The encoder takes the finished mesh (positions, faces, 48 SH coefficients per vertex, per-vertex opacity) and writes one file of five entropy-coded streams plus a small header. The lossy steps (SH dictionary assignment, position and DC quantisation, one opacity constant) all happen before the streams; the streams themselves are lossless and the encoder verifies a byte-exact round trip before writing.

1. **Connectivity.** Faces are visited in a cache-friendly strip order (meshoptimizer). The decoder keeps a ring buffer of the 64 most recently decoded *directed* edges. For each triangle the encoder

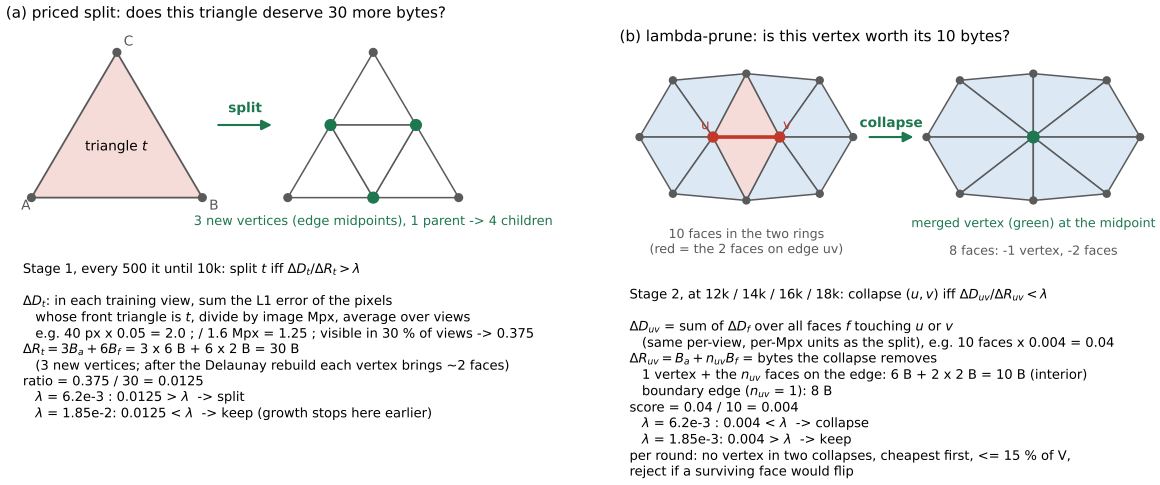


Figure 2: One priced split (a) and one λ-collapse (b), with illustrative numbers. Both use the same λ, the same per-face distortion and the same two byte prices B_a, B_f.

writes either *hit* (the triangle shares an edge with the buffer: one flag, the buffer slot and the third vertex) or *miss* (three vertices). A vertex seen for the first time is written as a single zero symbol, its id being implicitly the next unused one, so no vertex order is transmitted; a known vertex is written as the zigzag difference to the last id. All symbols go through an adaptive binary arithmetic coder [1] with 5×256 contexts and no transmitted tables, in the spirit of the mesh buffer of Deering [2]. The stream costs ≈ 15 bit per triangle (66 raw). Edgebreaker-class coders (≈ 2 bit per triangle on manifold meshes) do not apply directly to MeshSplatting's output (45 % boundary edges, thousands of components); MPEG TFAN on the same mesh needs 19.9 MB for geometry against our 16 MB (Section 2.3).

2. **Positions** are quantised per axis on a uniform grid of 2^b levels over $[\min, \max]$; b is chosen so that the grid step is one tenth of the median edge length of the trained mesh ($b = \lceil \log_2(\text{extent}/(0.1 \cdot \text{median edge})) \rceil$, extent = the largest bounding-box side), giving $b = 14$–18 per scene; on Room, $18 \to 14$ bits costs 0.06 dB and 13 bits starts flipping faces. Each vertex is predicted from its smallest-index already-decoded neighbour on the mesh graph (rebuilt from the decoded faces) and the integer residual is arithmetic-coded with 1536 contexts (predictor kind, axis, magnitude of the previous residual, bit position).
3. **SH dictionary indices.** The 45 non-DC coefficients of every vertex are replaced by an index into a main bank of $K = 4096$ codes plus a residual bank of $R = 256$. The banks are clustered by k-means (8 Lloyd iterations on a 65 k-vertex sample, then 4 full-set assignment / update rounds); with geometry and assignments frozen, the two banks and the per-vertex DC term are then optimised for 400 Adam steps against the training-view rendering loss (L1 + D-SSIM, as in training) with validation-based early stopping. This refit is worth +0.2 dB at $K = 16384$ and +0.6 dB at $K = 1024$. The two index arrays are coded with a 1-ring context: for vertex v the candidates are the distinct codes of its already-decoded neighbours (at most 32); a hit flag (context = number of candidates) followed by the candidate slot is written when v reuses one of them, otherwise the raw code under an adaptive order-0 model. Hit rates are 0.12–0.22 (vs 1/4096 for independent codes), saving about 0.5 MB per asset; the indices are otherwise close to incompressible (11.2 of 12 bits), which is why dictionary cost is controlled by K and λ rather than by entropy coding. The neighbour prediction is the discrete analogue of parallelogram attribute prediction [3, 4].
4. **DC.** The three degree-0 coefficients are uniformly quantised per channel to 2^7 levels over $[\min, \max]$, stored as one byte per channel and compressed with byte-shuffle + zstd [5] without spatial prediction (14 bit per vertex, 21 % of the file). Seven bits sit on the RD hull: on Room, 8 bits cost 1.3 MB for 0.03 dB (the same slope as the dictionary lever) and 6 bits lose 0.12 dB per MB saved.

Table 6 reports the 8-bit variant on every λ asset.

5. **Side data.** The two banks are stored as FP32 arrays over the codes actually used (180 B per code, 0.6–0.8 MB after zstd) together with the quantisation ranges and one opacity scalar for the whole mesh: in MeshSplatting's opaque stage every per-vertex opacity saturates (≥ 0.999), so we store their mean logit; the largest per-vertex deviation is 10^{-3}, the PSNR change is below 10^{-4} dB, and 1.7 MB are saved.

The decoder is a standalone program with no access to the training data: it decodes the five streams in this order, each conditioned only on what was decoded before, rebuilds positions, faces and per-vertex SH, and hands a standard mesh to any rasteriser. It runs in 2–6 s on one CPU core. For a Room asset at full growth the file is roughly 16 MB geometry (connectivity $\approx$ 12, positions $\approx$ 4) + 15 MB SH (indices, DC, banks); the SH dictionary is what makes a 740 MB FP32 model fit in 31 MB at -0.4 dB before any growth control. The position stream is already neighbour-predicted and context-coded; the remaining geometry redundancy is in the connectivity of a non-manifold mesh, i.e. in the mesh structure rather than in the entropy coder, and making the mesh manifold is future work.

2 Experiments

Setup. Mip-NeRF 360 (9 scenes) and Tanks & Temples (truck, train), test split = every 8th view. FP32 = our reproduction of MeshSplatting with the upstream flags (`-indoor` on room, counter, kitchen, bonsai); it matches the paper within ± 0.1 dB. MB = 10^6 bytes of the delivered file; ratio = FP32 size / delivered size. Each λ point is one training run plus one encode; nothing is tuned per scene. LPIPS in our delivered-asset rows is the pipeline convention (inputs in $[-1, 1]$), about 0.03 higher than the 3DGS convention used by the paper and by every published baseline; the unified evaluation pass will put all rows on the 3DGS convention.

2.1 Headline: the RD curve

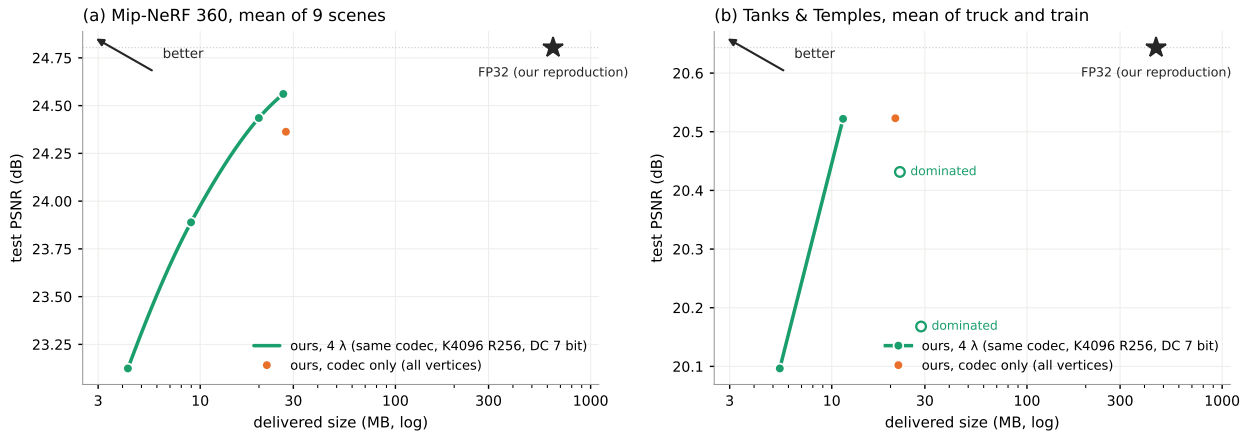


Figure 3: Dataset-mean RD curves behind Table 1: the four λ points (same codec, K4096 + R256, DC 7 bit), the codec-only point (all vertices kept) and the FP32 model. On T&T the two smallest-λ points are dominated (hollow): see Section 2.2.

2.2 Per scene

At $\lambda = 6.2 \times 10^{-3}$ the same price stops growth at 5.4–8.0 MB on the indoor scenes and 8.9–13.4 MB outdoors: the byte budget is an outcome of the scene content, not a preset cap. On flowers, treehill, truck and train the two smallest-λ points are dominated (a larger asset with no higher test PSNR): ΔD_t measures training-view error, and on these sparsely covered scenes the extra triangles reduce it without improving the test views (validation PSNR rises by 0.5–1.2 dB, test by ≤ 0.2). The Pareto

Table 1: Dataset means: Mip-NeRF 360 (9 scenes) and Tanks & Temples (truck, train). ΔPSNR is against the dataset's own FP32 row.

dataset	point	MB	ratio	PSNR / SSIM / LPIPS	ΔPSNR vs FP32
Mip-NeRF 360	FP32 (our reproduction)	642	1×	24.81 / 0.731 / 0.309	–
	$\lambda = 6.2 \times 10^{-4}$	26.6	24×	24.56 / 0.717 / 0.385	−0.24
	$\lambda = 1.85 \times 10^{-3}$	20.0	32×	24.44 / 0.709 / 0.405	−0.37
	$\lambda = 6.2 \times 10^{-3}$	9.0	72×	23.89 / 0.672 / 0.464	−0.92
	$\lambda = 1.85 \times 10^{-2}$	4.3	151×	23.12 / 0.624 / 0.518	−1.68
Tanks & Temples	FP32 (our reproduction)	459	1×	20.64 / 0.760 / 0.272	–
	$\lambda = 6.2 \times 10^{-4}$ (dominated)	28.7	16×	20.17 / 0.737 / 0.332	−0.47
	$\lambda = 1.85 \times 10^{-3}$ (dominated)	22.3	21×	20.43 / 0.745 / 0.331	−0.21
	$\lambda = 6.2 \times 10^{-3}$	11.4	40×	20.52 / 0.737 / 0.358	−0.12
	$\lambda = 1.85 \times 10^{-2}$	5.4	85×	20.10 / – / –	−0.54

Table 2: Per scene: FP32 model and the four λ points. Each cell is (MB (ratio), ΔPSNR vs the scene's FP32 row).

scene	FP32 MB	FP32 PSNR	λ 6.2e−4	λ 1.85e−3	λ 6.2e−3	λ 1.85e−2
room	597	28.48	(22.1 (27×), −0.23)	(12.4 (48×), −0.46)	(5.4 (110×), −1.18)	(2.9 (207×), −2.10)
counter	520	26.48	(20.7 (25×), −0.53)	(13.1 (40×), −0.64)	(6.4 (82×), −1.16)	(3.3 (155×), −1.94)
kitchen	507	27.46	(24.5 (21×), −0.37)	(16.5 (31×), −0.57)	(8.0 (63×), −1.49)	(3.8 (134×), −2.84)
bonsai	678	28.28	(24.3 (28×), −0.39)	(13.2 (52×), −0.87)	(6.1 (112×), −1.92)	(3.5 (192×), −2.92)
garden	740	24.75	(36.4 (20×), −0.02)	(24.6 (30×), −0.25)	(10.2 (73×), −1.10)	(5.0 (149×), −1.97)
bicycle	680	23.04	(31.6 (22×), −0.05)	(27.0 (25×), −0.02)	(11.9 (57×), −0.35)	(5.2 (131×), −0.96)
stump	698	24.99	(27.6 (25×), −0.18)	(21.4 (33×), −0.28)	(8.9 (79×), −0.87)	(3.9 (178×), −1.78)
flowers	668	19.45	(25.7 (26×), −0.43)	(24.2 (28×), −0.38)	(12.0 (56×), −0.24)	(5.6 (118×), −0.61)
treehill	692	20.39	(27.0 (26×), −0.08)	(27.4 (25×), +0.06)	(11.9 (58×), −0.01)	(5.1 (136×), −0.09)
truck	432	22.39	(27.3 (16×), −0.61)	(19.3 (22×), −0.19)	(9.4 (46×), −0.02)	(4.4 (98×), −0.75)
train	485	18.90	(30.1 (16×), −0.35)	(25.4 (19×), −0.24)	(13.4 (36×), −0.22)	(6.4 (76×), −0.35)
Mip-360 mean (9)	642	24.81	(26.6 (24×), −0.25)	(20.0 (32×), −0.37)	(9.0 (72×), −0.92)	(4.3 (151×), −1.69)
T&T mean (2)	459	20.64	(28.7 (16×), −0.47)	(22.3 (21×), −0.21)	(11.4 (40×), −0.12)	(5.4 (85×), −0.54)

hull is drawn through the non-dominated points. Section 2.5 shows the complementary fact: at equal vertex count, choosing the triangles in the loop beats choosing them afterwards.

2.3 Same asset, different codecs

Table 3: Room, same FP32 asset, different routes.

route	MB	PSNR	note
PLY + zstd	158.5	28.09	lossless after 8-bit SH quantisation
MPEG SC3DMC/TFAN (Open3DGC), SH as 8-bit attributes	108.0	≈28.09	strongest generic row; geometry 19.9 + SH 88.1
Draco, SH as attributes	456.1	28.09	splits vertices 4.5×
MPEG or Draco geometry + JPEG texture (4k / 8k atlas)	91–146	25.3	SH dropped, −3.0 dB
ours, codec only (level 1.0, no growth control)	30.9	28.06	3.5× smaller than MPEG
ours, $\lambda = 2 \times 10^{-3}$	13.7	27.99	7.9× smaller than MPEG

Of the 108 → 31 MB gain, about 73 MB comes from replacing per-vertex SH by dictionary indices and about 4 MB from our geometry streams: keeping our SH streams and replacing only our position + connectivity streams by MPEG SC3DMC/TFAN (19.9 MB vs our 16 MB) makes the file 13 % larger. The 108 MB row is MPEG end to end, with the SH coefficients as 8-bit attributes (88 MB). Figure 4 uses the 10/24 Room asset (trained without `-indoor`); every row in it is on that one asset, so it is internally consistent. Garden behaves the same way (MPEG 126 MB vs ours 32.5 MB at equal quality).

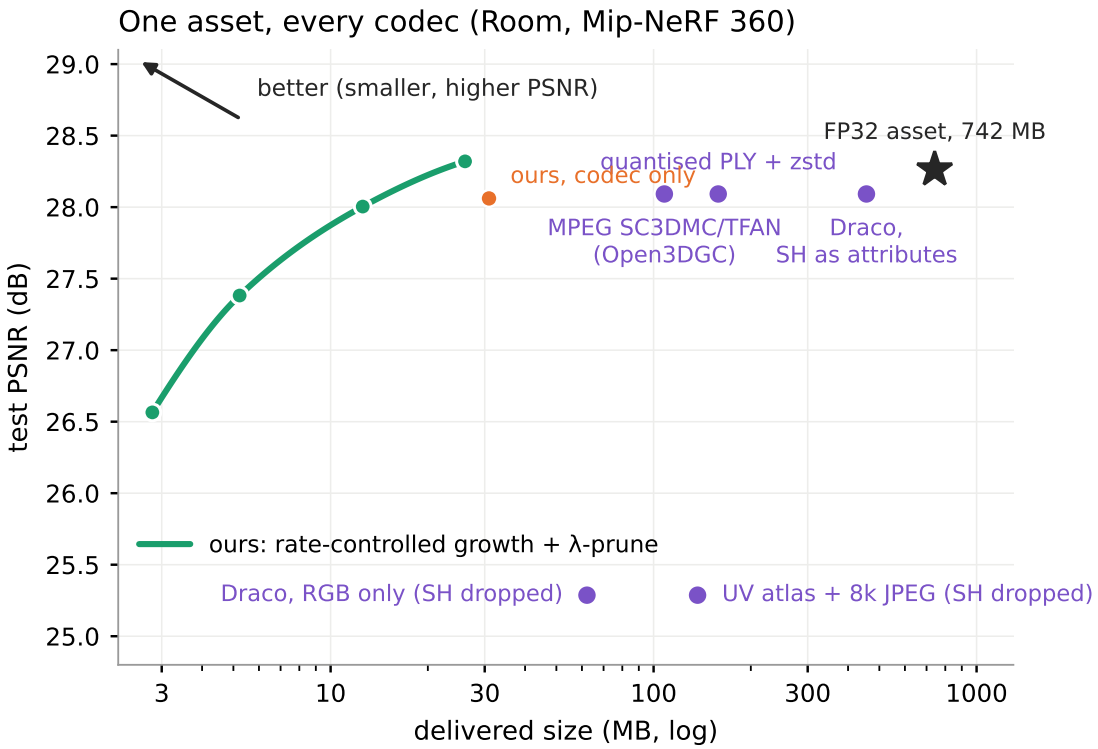


Figure 4: One FP32 Room asset (3.19 M vertices, 742 MB) through generic codecs and through ours. Every point is the same trained model; only the codec differs.

Table 4: Cross-method comparison, dataset means, engine-ready mesh methods. Published numbers verbatim; FPS as reported, hardware differs by row; "–" = not reported. † pipeline LPIPS convention, ≈ 0.03 higher than the 3DGS convention of the other rows. [a] RGB-only PLY, SH dropped. [b] plus a CPU decode of 2–6 s; FPS measured on Room (iPhone 17 Pro / MacBook Pro M2 Max, WebGL2).

method	Mip-360 PSNR / SSIM / LPIPS	MB	T&T PSNR / SSIM / LPIPS	MB	FPS
MeshSplatting, paper (2025)	24.78 / 0.728 / 0.310	100[a]	20.52 / 0.745 / 0.287	–	220 / 190 (M4)
MeshSplatting FP32, our reproduction	24.81 / 0.731 / 0.309	642	20.64 / 0.760 / 0.272	459	–
MiLo (SIGGRAPH Asia 2025)	24.09 / 0.688 / 0.323	253	21.46 / 0.706 / 0.348	–	170 / 160 (M4)
Binary Opacity Grids (SIGGRAPH 2024)	25.62 / – / –	913	–	–	927 (desktop)
RaDe-GS mesh (2024)	23.56 / 0.668 / 0.361	1100	20.51 / 0.659 / 0.344	–	OOM on M4
GOF mesh (2024)	20.78 / 0.573 / 0.465	1500	21.69 / 0.690 / 0.326	–	OOM on M4
ours, $\lambda = 6.2 \times 10^{-4}$	24.56 / 0.717 / 0.385†	26.6	20.17 / 0.737 / 0.332†	28.7	350 phone / 820 laptop[b]
ours, $\lambda = 1.85 \times 10^{-3}$	24.44 / 0.709 / 0.405†	20.0	20.43 / 0.745 / 0.331†	22.3	350 phone / 820 laptop[b]
ours, $\lambda = 6.2 \times 10^{-3}$	23.89 / 0.672 / 0.464†	9.0	20.52 / 0.737 / 0.358†	11.4	350 phone / 820 laptop[b]

2.4 Against other methods

Inside the engine-ready mesh class we reach MiLo's quality at 22× fewer bytes, RaDe-GS's at 175×, GOF's at 352× with +2.3 dB; the authors' own RGB-only PLY (100 MB) is 4× larger than our best point and 0.22 dB higher only because it drops SH entirely. Against 3DGS-class codecs our absolute PSNR is about 3 dB lower; this is the opaque-mesh representation gap, present already in uncompressed MeshSplatting (2.7 dB below 3DGS), not a codec gap. Published mesh methods do not compare against Gaussian codecs, and neither does the main text; the numbers are in Appendix B for reference.

2.5 Ablation: is the rate in the loop worth it?

With the codec held fixed, post-hoc edge collapse to 10 MB falls to 22.7 dB mean, training under a fixed vertex cap reaches 24.9 dB, and λ-priced growth plus prune reaches 25.0 dB and keeps going down to 4 MB. At matched vertex count the in-the-loop choice wins on every scene, by +0.3 to +4.3 dB. The prune rounds alone are a small effect (−8 % bytes at +0.01–0.02 dB on Room); the

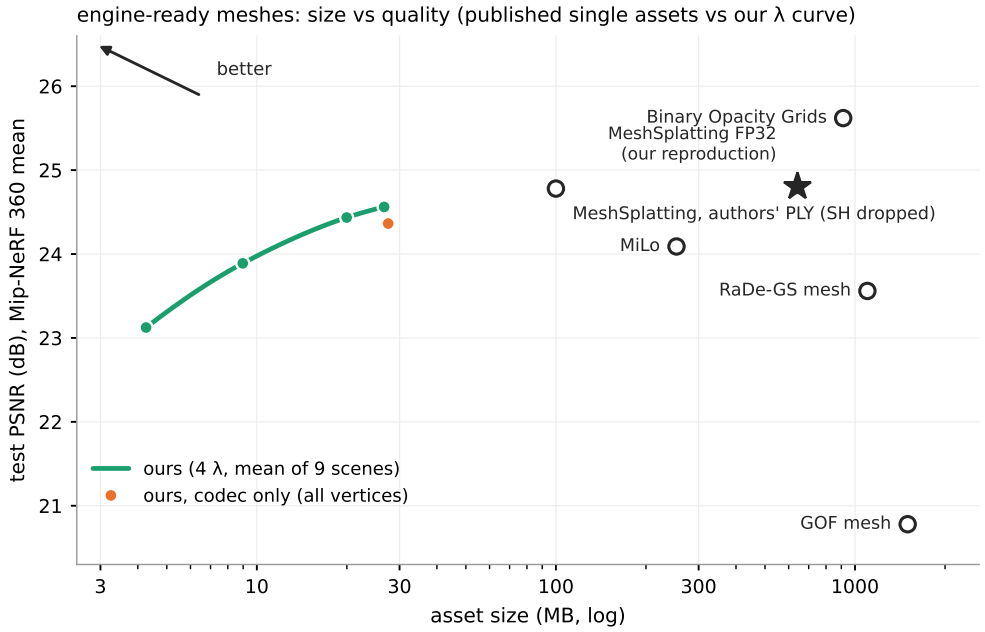


Figure 5: Engine-ready mesh methods, Mip-NeRF 360 means: size vs PSNR. Every published point is that method's own reported single asset (Table 4); none of them offers a rate control, so each has one point and ours is the only curve. The top-right point is Binary Opacity Grids (913 MB, 25.62 dB), which no opaque mesh of this class reaches at any size.

gain is in where growth is allowed to happen.

2.6 Deployability

Table 5: Deployability at the $\lambda = 6.2 \times 10^{-3}$ point. Laptop = MacBook Pro M2 Max (Chrome), phone = iPhone 17 Pro (Safari), WebGL2 viewer, SH degree 3, full capture resolution.

scene	asset MB	ratio	train min, upstream → ours	CPU decode s (1 core)	laptop ms (fps)	phone ms (fps)
room	5.2	142×	51 → 27 (0.52×)	2.3	1.2 (820)	2.8 (352)
garden	10.2	73×	49 → 24 (0.48×)	4.4	1.8 (566)	6.3 (160)
bicycle	11.9	57×	41 → 25 (0.60×)	5.4	1.9 (523)	6.9 (145)
11-scene mean	9.4	73×	0.57×	4.1	–	–

Training takes 0.57× the upstream time because the priced growth stops at fewer triangles and the rasteriser's cost scales with their number; nothing else in the optimisation changed. The decoder is a standalone single-core program with no access to the training data.

2.7 Joint λ× dictionary-size traversal

The λ curve above is traced at one delivery setting (K4096 + R256, DC 7 bit) chosen on post-hoc vertex levels of Room (Appendix A). To show that this setting stays on the joint hull, every λ asset of room, kitchen and garden is re-exported, without training, with four other dictionaries ($K \times R \in \{4096 \times 1,\ 1024 \times 256,\ 4096 \times 1024,\ 16384 \times 1024\}$) and with DC at 8 bits.

2.8 What is still open

- Table 6 (the λ× dictionary fan and the DC 8-bit column, 12 λ assets + the room/kitchen FP32 deliveries) is being filled; it also answers whether the 0.5–0.75 dB indoor delivery loss is a dictionary or a DC effect.

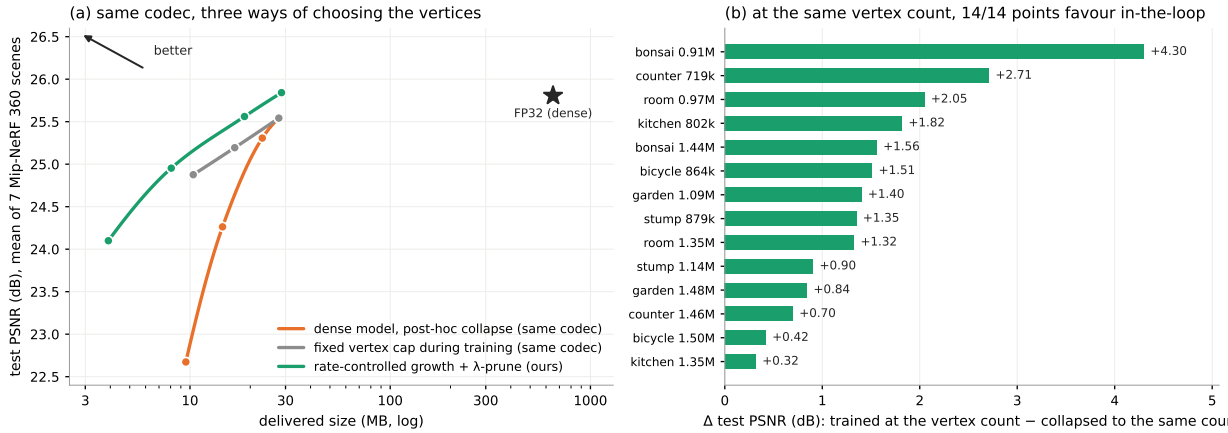


Figure 6: Same codec, three ways of choosing vertices (7 Mip-NeRF 360 scenes, older parameter set): post-hoc edge collapse, fixed vertex cap, λ-growth + prune. (b) At equal vertex count the in-the-loop choice wins on 14 of 14 points.

Table 6: $\lambda\times$ dictionary fan (room, kitchen, garden; every cell MB / ΔPSNR vs that λ's K4096 + R256 + DC 7-bit row). [**Table X: filled from `lambda-k-fan-v1.txt` and `indoor-diag-v1.txt` when the re-export jobs finish.**]

scene, λ	DC 8 bit	K4096 R1	K1024 R256	K4096 R1024	K16384 R1024
pending					

- The byte prices are a calibrated constant proxy for the rate; a context-dependent (trainable) rate model would make B_a a function of the local neighbourhood. We report the sensitivity of the operating point to the constants (Section 1.2) and leave the trainable model as future work.
- Unified evaluation on the 3DGS LPIPS convention ($\approx$ 3 GPU-h) and MPEG rows for the remaining nine scenes (CPU) are pending; neither changes the ordering above.

A Dictionary size on post-hoc vertex levels

On Room the $K \times R$ sweep was run at four vertex levels (100, 80, 70, 50 % of the dense mesh, 12 dictionaries each) and on six scenes at one level; a greedy allocator over the three levers (vertices, dictionary, precision) reproduces the global Room hull on 9 of 9 points. Near the chosen setting every lever costs 0.02–0.06 dB per MB: K4096 R256 $\rightarrow$ R1024 buys 0.022 dB per MB, DC 7 $\rightarrow$ 8 bit 0.022, K16384 0.014–0.020; the frontier is smooth and no single knob dominates, which is why one setting is used for every λ and Table 6 checks it on the λ assets themselves.

B 3DGS-class methods, for reference

References

[1] I. H. Witten, R. M. Neal, J. G. Cleary. Arithmetic coding for data compression. *Comm. ACM* 30(6), 1987.

[2] M. Deering. Geometry compression. *SIGGRAPH* 1995.

[3] C. Touma, C. Gotsman. Triangle mesh compression. *Graphics Interface* 1998.

[4] Google. Draco 3D data compression. https://github.com/google/draco.

[5] Y. Collet, M. Kucherawy. Zstandard compression and the application/zstd media type. RFC 8878, 2021.

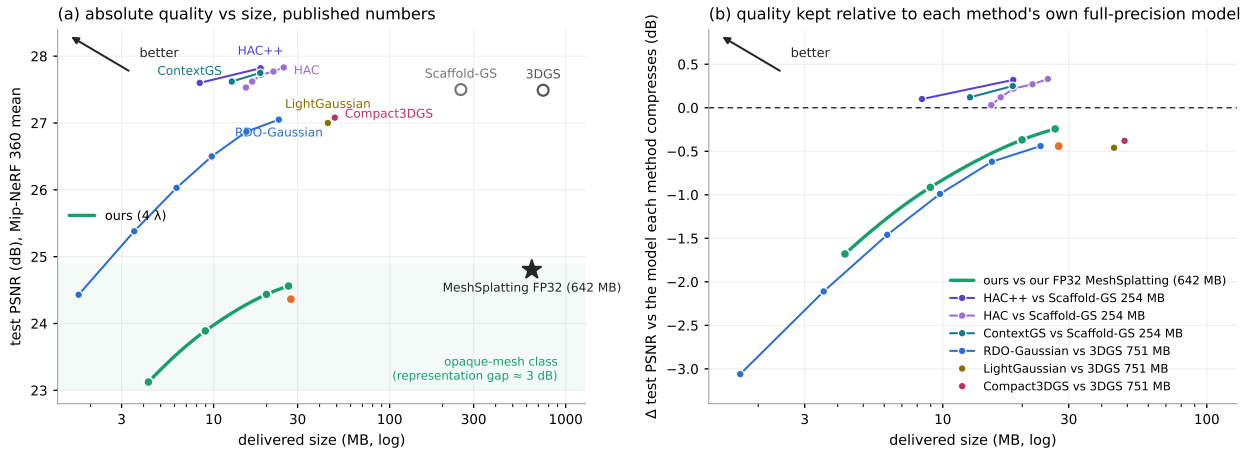


Figure 7: 3DGS-class codecs. (a) absolute numbers; (b) quality retained relative to each method's own full-precision model.

Table 7: Cross-method comparison, dataset means, 3DGS-class methods (not engine-ready meshes). Published numbers verbatim; FPS as reported, hardware differs by row; "–" = not reported. Ours (Table 4) is ≈ 3 dB below this class in absolute PSNR; this is the opaque-mesh representation gap, present already in uncompressed MeshSplatting.

method	Mip-360 PSNR / SSIM / LPIPS	MB	T&T PSNR / SSIM / LPIPS	MB	FPS
3DGS (SIGGRAPH 2023)	27.46 / 0.812 / 0.222	750.9	23.69 / 0.844 / 0.178	431.0	99 (desktop)
Scaffold-GS (CVPR 2024)	27.50 / 0.806 / 0.252	253.9	23.96 / 0.853 / 0.177	86.5	135
HAC++ low (2025)	27.60 / 0.803 / 0.253	8.34	24.22 / 0.849 / 0.190	5.18	130–151
HAC++ high (2025)	27.82 / 0.811 / 0.231	18.48	24.32 / 0.854 / 0.178	8.63	130–151
RDO-Gaussian (ECCV 2024)	27.05 / 0.802 / 0.239	23.46	23.34 / 0.835 / 0.195	12.03	–
Compact3DGS (CVPR 2024)	27.08 / 0.798 / 0.247	48.80	23.32 / 0.831 / 0.201	39.43	–
LightGaussian (NeurIPS 2024)	27.00 / 0.799 / 0.249	44.54	22.83 / 0.822 / 0.242	22.43	–